A METHODOLOGY REPORT · 2026

From Friction to Value

How to find hidden value in an organization's processes and capture it by building internal tools with human–AI collaboration: a complete, field-tested method from diagnosis to realized return.

AUTHOR

Valentin Angenot

DOMAIN

Strategy · Engineering · AI

FOREWORD

Why this report exists

> *"The constraint on creating value inside most organizations has quietly shifted. It is no longer the cost of building software. It is the discipline to find what is worth building."*

For most of the last two decades, the bottleneck on internal improvement was engineering capacity. A good idea for an internal tool died in a backlog because building it required a team, a budget, and a quarter. That economics has changed. A capable individual, working with a modern AI assistant, can now turn a clear specification into a working, production-grade tool in days. The marginal cost of software has collapsed.

But something has not changed, and has arguably become more valuable: the human work that surrounds the code. Understanding a process deeply enough to know where the waste hides. Sizing the opportunity in money so it earns a decision. Designing for a five-year life rather than a one-day demo. Bringing a team along so the tool is actually used. That work is judgment. And judgment does not automate.

This report sets out a complete method for doing both halves well. It is grounded in a single real deployment, deliberately anonymized, in which I, working with Claude, built a tool that is projected to save an organization close to six hundred thousand euros over ten years, at effectively zero infrastructure cost and in roughly ten working days. Every framework here was used in practice; none is theoretical.

It is written for the operator, the analyst, the consultant, and the manager who suspects there is value trapped in their own organization's daily routines and wants a rigorous, repeatable way to release it. Read it as a field guide, not a manifesto.

Valentin Angenot, 2026

EXECUTIVE SUMMARY

The argument and the result

Situation

A public-sector operator decided to bring a field operation in-house rather than continue outsourcing it. The financial logic was sound. But a cost buried in the administrative layer threatened to erase the margin: every job required a permit, and across the territory 24 separate administrations each imposed their own forms, platforms, deadlines, contacts, and fees. There was no standard. There was only a maze.

Complication

Prepared by hand, a single permit dossier consumed roughly 99 minutes of skilled time: searching for the right procedure, re-keying data across systems, drawing a compliant plan, drafting messages to multiple authorities. It also carried a real risk of rejection, which triggered an expensive re-work loop. The knowledge required to navigate all 24 procedures lived in one person's head, making the cost both a recurring drain and a key-person risk.

Resolution

Instead, I built a single browser-based tool in approximately ten working days, working with Claude. The operator now fills one form; the tool detects the relevant administration, applies its specific rules, and generates every required document and message automatically. For the signage plan, the operator designs it using a built-in visual editor on a real map, then the tool formats and exports it to the authority's specifications. Preparation time fell to 11 minutes. The tool runs entirely client-side, with no server, no IT budget, and no recurring cost.

```
99 → 11 min/task    ~88% reduction    €597K 10-year value    ≈ 150× return
```

On conservative assumptions (88 minutes saved per job, a fully-loaded labour cost of €77 per hour, 5,346 jobs over a ten-year horizon), the tool creates **€596,881 in administrative savings** alone, before accounting for fewer rejected applications, faster onboarding, and the strategic value of de-risking the entire internalization.

The transferable method

The tool is incidental. What generalizes is the discipline that produced it: assess the process until the friction is visible and timed; discover and size the value until it is expressed in money; design for the whole life of the tool; build fast with AI; co-construct

for adoption; deploy for sustainability; measure what was actually realized; and scale the approach.

> *Find expensive friction · size it in money · design for the long run · build fast with AI · co-construct for adoption · deploy for free · measure the return · repeat. Everything that follows is the detailed version of that sentence.*

PART I

I
Business Process Assessment

You cannot improve what you have not measured, and you cannot measure what you have not mapped. Before any tool is conceived, the existing process must be made visible, timed, and costed.

1.1 — Why value hides inside processes

Friction is invisible because it is distributed. No single instance is expensive; it is the repetition that compounds.

A task that takes ninety minutes once is a non-event. The same task performed several hundred times a year, by skilled staff, across an unstandardized landscape, is a structural cost, but it never appears as a line item. It is absorbed into salaries, hidden behind "that's just how it works," and protected by the fact that the person who performs it has quietly become the only person who can. Undocumented process friction is simultaneously a recurring cost and a key-person risk, and the second is often the more dangerous.

1.2 — The four-lens diagnostic

EXHIBIT 1 **The four-lens process diagnostic**

01 · MAP	02 · MEASURE	03 · DIAGNOSE	04 · QUANTIFY
Document every step, actor and handoff in a swimlane.	Time each step. Use ranges, not points, and flag conditional tasks.	Classify each step: value-adding, necessary, or pure waste.	Convert wasted time to money: minutes × loaded rate × annual volume.
→ Process map	*→ Time-and-motion sheet*	*→ Waste register*	*→ Cost-of-friction figure*

1.3 — Lens one: map the work

Begin with a **swimlane map**. Each actor (operator, manager, external authority) gets a lane, and the task flows left to right across them. A useful complement is the **SIPOC** frame (Suppliers, Inputs, Process, Outputs, Customers), which bounds the process and names who depends on it.

1.4 — Lens two: measure honestly, in ranges

Estimate the duration of each step as a range, not a single number. Three data-collection methods, in increasing order of reliability:

- **Self-estimate**: ask the person who does the work. Fast, but biased.
- **Structured walk-through**: sit with them through one real case and time each step. The default method.
- **Sampling**: log durations across several real cases. Most reliable; reserve for high-stakes baselines.

1.5 — Lens three: diagnose value versus waste

- **Value-adding**: the customer or organization would pay for this step.
- **Necessary but non-value-adding**: required by compliance or hand-off, but invisible to the beneficiary.
- **Pure waste**: searching, re-formatting, re-work after rejection, waiting. This is the primary target.

1.6 — Lens four: quantify in money

> COST-OF-FRICTION FORMULA
>
> ```
> Annual friction cost = (minutes of waste per task ÷ 60) × loaded hourly rate × annual task volume
> ```

Use the **loaded rate**: salary plus employer charges plus overhead, not the gross salary, and use realistic volumes. Understating both is how credible business cases are built.

1.7 — The output: a friction register

The assessment closes by consolidating findings into a single register, one row per source of friction, each carrying its quantified annual cost, its key-person risk, and a first

feasibility read. This register is the raw material of Part II.

PART II

II
Value Discovery & Targeting

An assessment surfaces many sources of friction. Resources are finite. This part is about choosing. It turns a register of pain points into a prioritized, financially-sized opportunity worth building for.

2.1 — A taxonomy of value

EXHIBIT 4 **The four forms of value**

FORM	DESCRIPTION	HOW TO PRICE
Hard savings	Direct labour time recovered.	Fully quantifiable
Risk reduction	Fewer errors and rejections; removal of key-person risk.	Price via expected loss
Capacity unlocked	Freed time redeployed to higher-value work; faster onboarding.	Price partially
Optionality	De-risks a larger initiative; creates a reusable capability.	Flag qualitatively

2.2 — Prioritization: the impact–feasibility matrix

Plot every opportunity on two axes: value at stake and feasibility of capture. The upper-right quadrant is where you start. The trap is the upper-left: seductive, high-value problems that are not yet feasible.

2.3 — Stress-testing the number

EXHIBIT 7 **Sensitivity of 10-year value to key assumptions**

SCENARIO	MIN. SAVED	JOBS / 10Y	RATE	10-YR VALUE
Pessimistic	70	4,000	€70	**€327K**

Conservative (base)	87	5,346	€77	**€597K**
Central	112	5,346	€77	**€769K**
Optimistic	138	6,000	€80	**€1.10M**

2.4 — Build, buy, or leave it

Buy when a mature product solves the process at a lower total cost than building it internally. **Build with AI** when the problem is specific, domain knowledge is the scarce input, and the internal build cost is lower than the price, rigidity, or lock-in of available solutions. **Leave it** when the value at stake does not justify either path.

The reference case was a strong build candidate because the rules were local, non-standard and frequently changing, while the AI-assisted build cost was extremely low compared with the expected value created.

2.5 — The business case on one page

The output of value discovery should not be a long document. It should be a one-page decision case built on the pyramid principle: answer first, then support. The point is not to show all the analysis; it is to make the decision obvious.

A good one-page business case answers six questions in order: what should we do, why now, how much value is at stake, what it will cost, what could go wrong, and what decision is required. Everything else belongs in the appendix.

EXHIBIT 8 **One-page business case structure**

BLOCK	QUESTION ANSWERED	CONTENT TO INCLUDE
1. Recommendation	What should we approve?	Build, buy, or leave; the chosen option and the one-sentence rationale.
2. Problem	What friction are we removing?	Current process, pain point, volume, wasted time, key-person or quality risk.
3. Value case	What is the prize?	Baseline time, time saved, loaded rate, annual volume, risk reduction, 10-year value.
4. Cost and effort	What does capture require?	Build or purchase cost, deployment effort, maintenance burden, ownership model.

5. Risks and safeguards	Why might value not materialize?	Adoption, data quality, rule changes, security, dependency risk, mitigation plan.
6. Decision ask	What exactly is needed now?	Approval for pilot, owner, timeline, budget, access, or go/no-go decision.

The discipline is subtraction. Keep the page focused on decision-critical information only. A CFO does not need every timing assumption on page one; they need the conservative value range, the cost to capture it, and the conditions under which the case fails.

```
"Turn this opportunity into a one-page business case for a senior decision-maker. Use six blocks: recommendation, problem, value case, cost and effort, risks and safeguards, decision ask. Keep the recommendation at the top and move all non-critical detail to an appendix list."
```

PART III

III
Designing Intelligently

A tool that works on the day it ships but cannot be maintained six months later destroys more value than it creates. Good design is mostly decided before any code exists.

3.1 — Optimize for total cost of ownership

The build is a one-off. Operation lasts years. Design for the years.

The right objective is **total cost of ownership**: build effort plus the cumulative cost of running, updating, fixing, and handing over the tool across its life. A design that saves five hours today but needs a developer every time a phone number changes has failed.

3.2 — Maintainability: separate data from logic

The single highest-leverage design decision is to externalize everything that changes from everything that does not. In the reference tool, the 24 rule-sets live in a plain JSON file, entirely separate from the application code. A non-developer updates a contact by editing one line of text.

3.3 — Modularity: keep the codebase navigable

When you build with AI in a conversational loop, the natural output is one giant file. The model adds function after function to the same document, and before you realize it, you have 8,000 lines of JavaScript with no structure. It works today. Six months from now, nobody (including you) can find anything in it.

This is the second most important design decision, right after data-logic separation. You need to decide the module structure **before** the first build pass, not after.

The rule: one file, one job

Each file should do one thing and be readable on its own. For the reference tool, the codebase was structured around small, named modules rather than a single monolithic script:

EXHIBIT **Module structure of the reference tool**

FILE	RESPONSIBILITY	APPROX. SIZE
`index.html`	Static page shell: layout, containers, script loading	~220 lines
`styles.css`	Visual system, print styles, responsive layout	~650 lines
`app.js`	Application state, navigation, event orchestration	~480 lines
`forms.js`	Form schema, dynamic fields, data serialization	~420 lines
`rules.js`	Commune detection, rule resolution, fee/deadline logic	~360 lines
`validation.js`	Date checks, required fields, coherence controls	~280 lines
`editor.js`	Visual signage editor: map layer, drag-and-drop, placement UI	~780 lines
`signage.js`	Sign catalogue, grouping logic, plan export data	~430 lines
`documents.js`	Generated Word documents and annex formatting	~520 lines
`emails.js`	Recipients, subjects, message bodies, attachment list	~240 lines
`storage.js`	Autosave, local history, browser persistence	~210 lines
`communes.json`	All 24 rule-sets: contacts, deadlines, formats, fees	Data only

The important point is not the exact number of files; it is the boundary between responsibilities. A future maintainer should know where to look before opening the code. If a commune contact changes, they edit `communes.json`. If an email template changes, they open `emails.js`. If a validation rule fails, they inspect `validation.js`, not a 7,000-line application file.

Even within a module, internal structure matters. Group related functions together. Use clear comment headers to mark sections (`// ═══ DOCUMENT GENERATION ═══`). Keep utility functions at the bottom. When a function exceeds 80 lines, it is doing too much.

How to prompt for modularity

The key is to tell the model the module boundary at the start of each conversation, not to ask it to refactor later.

```
"This file handles only document generation. It exports two functions:
generatePermitDoc(data) and generateDeviationDoc(data). It does not touch the
DOM, does not access localStorage, and does not depend on any global variable.
Build generatePermitDoc first."
```

When a file has grown too large and you want to split it, do not ask the model to "refactor everything". That produces chaos. Instead:

```
"Extract the five functions related to email generation (listEmailRecipients,
buildEmailBody, buildEmailSubject, attachFiles, formatContactList) into a new
file called email.js. The main file should import and call them. Change nothing
else."
```

Signs that your codebase needs restructuring

- You scroll for more than ten seconds to find a function.
- The model starts producing inconsistent output because the file is too long for its context window.
- Two features have nothing in common but share a file.
- You are afraid to touch one function because it might break something elsewhere in the same file.

Restructuring is always cheaper now than later. The cost of splitting a 5,000-line file into three clean modules is one conversation. The cost of debugging an 10,000-line monolith for years is much higher.

3.4 — Operability: run where the work happens

The reference tool runs entirely in the browser, client-side, with no server. The principle generalizes: **choose the simplest deployment that reaches the actual user in their actual context.**

3.5 — Resilience: fail safe, degrade gracefully

External dependencies fail; the design must assume it. Where the tool calls an external service, it must surface a non-blocking message and let the user proceed manually.

3.6 — Pre-build design checklist

- Is everything that changes separated from the code?
- Does the tool run in the user's real context?
- What happens when each external dependency fails?
- Are inputs validated to prevent the most expensive downstream error?
- Could a non-developer make routine updates?

- Is the simplest viable architecture chosen?
- Is the module structure decided before the build starts? Could someone new find any function in under ten seconds?

PART IV

IV
Building with Claude

With a sized opportunity and a sound design, the build becomes fast. The leverage comes from a clear division of labour: human judgment, AI throughput.

4.1 — The collaboration model

AI did not replace me as an engineer. It collapsed the distance between knowing what to build and having it built.

And the same logic applies to the end user. The tool does not replace the field operator. It upgrades them. Before, a skilled worker spent most of their permit time buried in chaotic admin: hunting for the right form, cross-referencing contacts, reformatting the same data across three systems. The actual expertise (reading a site, understanding traffic flow, deciding where to place signs) was squeezed into a fraction of the process. Now that same person spends their time designing compliant signage plans with a purpose-built visual editor, making real decisions about real roads. The tool absorbed the bureaucracy so the human could do the thinking.

EXHIBIT 12 **Division of labour**

HUMAN — JUDGMENT	CLAUDE — THROUGHPUT
Domain knowledge & field research	Specification → working code
Deciding what to build and why	Architecture options & trade-offs
Defining "correct" and edge cases	Document-generation engine
Stakeholder relationships	Refactoring & debugging
Final validation & sign-off	Iterating at conversational speed

The scarce input is on the left, which is why I could do it alone.

4.2 — Prompting as delegation

Good prompting is good delegation. Six principles:

1. **Lead with context, then the task.** State who, what, and why before the specific request.
2. **Treat constraints as information.** Framework, format, allowed dependencies, performance context.
3. **Iterate in vertical slices.** Build a thin working version end-to-end, then deepen.
4. **Ask for reasoning on high-stakes calls.** Request options and trade-offs before committing.
5. **Push back precisely.** "Too complex. I want zero dependencies" corrects faster than vagueness.
6. **Show, don't only tell.** A reference document replaces paragraphs of description.

4.3 — A worked prompt sequence

Pass 1 — Frame & skeleton

```
"I'm building a browser-only permit tool for field operators across 24
administrations. They're on tablets with poor connectivity, so no server, zero
dependencies. Build one form capturing every field any administration might
need."
```

Pass 2 — Vertical slice

```
"Now detect the administration automatically from the postal code, and load its
rules from a separate JSON file so a non-developer can edit them. Show me the
data structure first."
```

Pass 3 — Deepen & harden

```
"Generate the official document as a Word file, client-side. Add validation:
block submission if dates are incoherent. If geocoding fails, let the user
enter the address manually. Never block them."
```

Pass 4 — Refine against reality

```
"I ran a real past case through it. The signage list should be grouped by sign
type with counts, not listed individually. Fix just that."
```

4.4 — The build loop

Specify one slice → **Generate** with Claude → **Test & review** against reality → **Refine** or deepen. Each loop leaves a working, demonstrable tool.

4.5 — Quality control

- **Critical paths**: line-by-line review: document generation, validation logic.
- **Real-case testing**: run past dossiers through the tool; compare outputs.
- **Spot review**: scan; ask Claude to explain anomalies.

PART IVB

IVb
Advanced Prompting & Token Strategy

Part IV covered the basics. This part goes deeper into prompting techniques and into the economics of tokens, which determine whether AI-assisted building stays cheap or becomes expensive.

4b.1 — The anatomy of a good prompt

Every effective prompt has three layers: **context** (who you are, what exists), **task** (the specific thing you want done), and **constraints** (what not to do, what format to use). Omitting any layer forces the model to guess, and guesses cost rework.

4b.2 — Prompting strategies for coding, in practice

The list below is not theoretical. Every technique was used during the reference build. Each one includes the concrete prompt pattern that makes it work.

1. Frame the project before the first line of code

The model cannot infer your architecture, your users, or your constraints. State them up front, once.

```
"I am building a client-side permit management tool in vanilla HTML/JS. It runs on tablets with unreliable connectivity. No frameworks, no server, no npm. All data comes from a local JSON file. The user is a field operator who is not technical. Keep the UI simple and the code readable."
```

2. Build in vertical slices, not horizontal layers

Do not ask for "the database layer" and then "the UI layer". Ask for one thin feature that works end-to-end. Test it. Then ask for the next.

```
"Build a form with fields for: street address, start date, end date, commune name. When the user types a commune name, auto-suggest from a list in the JSON file. That is all for now."
```

3. Stack constraints before the task

The model respects constraints it sees early. Constraints placed after the request are often ignored.

```
"Constraints: no external libraries, no fetch calls, all logic in a single
file, no inline styles. Now add a function that exports the current form data
as a .docx file using only the docx.js library already loaded in the page."
```

4. Ask for the data structure before the code

When a feature requires structured data, ask the model to propose the schema first. Review it. Then ask for the implementation. This prevents expensive rewrites.

```
"I need to store the rules for 24 communes: each has a name, a postal code
range, a contact email, a deadline in working days, and a list of required
documents. Propose a JSON schema for this. Do not write any code yet."
```

5. Show, do not describe

When you want the output to match a specific format, paste a real example. The model matches patterns better than it follows abstract descriptions.

```
"Here is what the generated email should look like: [paste real example]. Now
build a function that takes form data and produces an email body matching this
exact format and tone."
```

6. Request diffs, not full rewrites

Once the codebase is large, never ask the model to regenerate the entire file. Specify the exact function to change. This saves tokens and prevents the model from accidentally breaking working code.

```
"In the generateDocument() function, add a check: if the detour checkbox is
checked, insert a detour section after the signage table. Change only that
function. Keep everything else exactly as it is."
```

7. Correct with precision, not politeness

When the output is wrong, describe exactly what happened and what should have happened. The more specific you are, the fewer iterations it takes.

```
"The signage list renders each sign individually. It should group signs by type
and show a count next to each type. Example: 'B14 x3, C27 x1, KC1 x2'. Fix only
the renderSignageList function."
```

8. Use checkpoints to catch drift

After several rounds of edits, the model may lose track of the full file state. Periodically ask it to audit itself.

```
"List every function in this file, what it does, and which other functions call it. One line per function. Do not change anything."
```

9. Harden by naming the failure mode

Do not just say "handle errors". Name the specific failure and describe exactly what should happen.

```
"When the geocoding API returns a 429 or a network error, do not block the user. Show a yellow banner saying 'Geocoding unavailable, enter the address manually' and reveal a manual address input field. The form must remain fully usable."
```

10. Ask for a rationale before committing

For decisions with long-term consequences (architecture, data model, dependency choices), ask the model to present options with trade-offs. Then you decide.

```
"I need to store submission history so the user can recall past permits. Give me three options for how to do this in a client-side tool, with the trade-offs of each. Consider localStorage limits, data size, and what happens when the user clears their browser."
```

11. Separate "what" prompts from "how" prompts

First get agreement on the requirement. Then ask for the implementation. Mixing both in one prompt leads to solutions that miss the point.

```
Step 1: "When a commune has a specific email template, the generated email should use that template instead of the default. Does this mean I need a 'template' field in the commune JSON?" Step 2: "Yes, add it. Now implement the logic in the sendEmail() function."
```

12. Start a checkpointed conversation when context gets stale

If the model starts forgetting earlier instructions, ignoring constraints, or producing inconsistent output, the conversation is probably carrying too much stale context. Do not simply open a blank chat and hope it remembers the project. Start from a checkpoint: paste or upload the latest file, add a short handover note, and state the next change precisely.

4b.3 — Token economics

Tokens are the unit of AI consumption, roughly 0.75 words per token in English, slightly more in French.

What counts as tokens

- **Input tokens**: everything you send: system prompt, conversation history, pasted code, current message.
- **Output tokens**: everything Claude writes back.
- **Context window**: the maximum combined input + output the model can process in one call. The limit depends on the model, product, and access path; verify against current provider documentation before making cost or architecture decisions.

The hidden cost: stale context

The real cost is not only the number of tokens. It is the quality of the context the model is forced to reason over. In API-style conversations, previous turns accumulate inside the context window. In chat products, the exact mechanics may include rolling context, caching, or compaction. Either way, very long sessions eventually mix useful project state with outdated attempts, abandoned decisions, and irrelevant debugging traces.

Starting a fresh conversation helps only if you carry forward the **current working state**. For code, that means pasting or uploading the latest file, opening the relevant repository in an agentic tool, or providing a compact handover note. Otherwise the model loses the exact state of the application and may rebuild from memory instead of modifying the real code.

- **Checkpoint after each stable feature.** Save the latest files and summarize the current architecture, decisions, unresolved bugs, and next step.
- **Start fresh from the checkpoint, not from a blank page.** Example: "Here is the current file and the handover note. Modify only the export function."
- **Use Projects, repo context, or system instructions for stable context.** Keep coding conventions and architecture rules there, but still provide the latest file when the task depends on exact code.

4b.4 — Eight token optimization techniques

1. **Be concise in prompts.** Every unnecessary word is a token.
2. **Ask for diffs, not full files.** "Show me only the changed function." Saves 90%+ of output tokens.

3. **Chunk large requests.** Five focused prompts beat one monolithic one.
4. **Avoid re-explaining stale context.** Reuse stable project instructions, but attach only the current file or relevant function when the edit depends on exact code.
5. **Use structured output formats.** JSON or code directly avoids prose wrappers.
6. **Set explicit length expectations.** "In two sentences" or "Under 50 lines."
7. **Prune pasted content.** Only the relevant section, not the entire file.
8. **Choose the right model for the task.** Haiku for formatting; Sonnet for building; Opus for architecture.

4b.5 — Cost benchmarks

The reference tool was built via Claude.ai Pro. Had it been built through the API using a Sonnet-class model, the estimated cost would be roughly in the tens of euros (before caching, batching, or subscription effects). The exact figure depends on the model and pricing at the time. The tool creates €597K of value. The cost of AI itself is, for practical purposes, a rounding error. The expensive input is the ten days of human judgment.

PART IVC

IVc
Understanding Claude

You do not need to understand how an engine works to drive a car. But understanding a few things about the model you work with makes you a dramatically better driver.

4c.1 — What Claude is

Claude is a large language model (LLM) built by Anthropic. It processes text and images as input, and generates text as output. It does not browse the internet during a conversation (unless given search tools), does not remember previous conversations (unless memory is enabled), and does not execute code by itself (unless given a code execution environment).

4c.2 — The model family

EXHIBIT A **Claude model tiers (as of May 2026; verify current names, sizes and pricing at anthropic.com)**

TIER	BEST FOR	CONTEXT	COST TIER
Haiku (fastest)	Fast, cheap tasks: formatting, boilerplate, data transforms	200K+	Lowest
Sonnet (balanced)	Coding, analysis, writing, most build tasks	200K+	Mid
Opus (most capable)	Complex reasoning, architecture, nuanced judgment	200K+	Highest

4c.3 — Context windows

The context window is the model's working memory. At 200,000 tokens, Claude can process roughly 500 pages in a single call. But in a long conversation with large code files, it fills faster than you expect.

- A 3,000-line JS file ≈ 15,000 tokens. Pasting it + prompt + modified output ≈ 35,000 tokens per round-trip.
- After 20–30 messages, accumulated history can consume 50–100K tokens.
- **Rule of thumb**: if the model seems to "forget" earlier instructions, start a fresh conversation.

4c.4 — Strengths and weaknesses

Strengths

- Turning a clear specification into working code, often on the first attempt.
- Refactoring large files while preserving behaviour.
- Generating boilerplate and templated content at speed.
- Explaining trade-offs between architectural options.

Weaknesses

- Inventing plausible-looking but incorrect answers ("hallucination").
- Maintaining consistency across very long conversations (context drift).
- Knowing the current state of external systems without being told.
- Resisting a user's incorrect premise.

The mitigation is always the same: **human review on critical paths** and **testing against real data**.

4c.5 — Access paths

- **claude.ai**: the chat interface. Best for interactive build sessions.
- **Claude API**: programmatic access. Pay per token.
- **Claude Code**: command-line agentic coding.
- **Claude Projects**: persistent context spaces with reference documents.

PART V

V
Co-Construction & Adoption

The graveyard of internal tools is full of technically excellent software that nobody used. Adoption is not a launch event; it is a design input present from the first day.

5.1 — Tools fail on adoption, not technology

The hardest problem is rarely making the tool work. It is making people want to use it.

The antidote is **co-construction**: building *with* the eventual users rather than *for* them. A tool used by 40% of the team captures, at best, 40% of the projected return.

5.2 — Five conditions for adoption

Awareness → **Desire** → **Ability** → **Proof** → **Reinforcement**. Skipping one stalls the whole. Co-construction feeds every stage, especially Desire and Proof, the two stages mandates cannot buy.

5.3 — Adoption levers across the build

EXHIBIT 16 **Adoption levers**

PHASE	CO-CONSTRUCTION ACTION	ADOPTION EFFECT
Assessment	Users validate the process map	Problem ownership
Design	Users review the proposed workflow on paper	Early objections surface
Build	Users see working slices each loop	Solution ownership
Pilot	A small team uses it on real work	Champions emerge
Roll-out	Champions train peers	Peer credibility beats mandate

5.4 — Documentation that outlives the builder

> *Knowledge encoded in a system and a document is an asset; knowledge in one head is a liability, the very risk the tool was built to remove.*

A final advantage of building with Claude is that the same assistant can help create the technical documentation needed for maintenance and handover. Once the application works, paste the final codebase, data schema and deployment setup into a documentation pass. Claude can turn them into a structured handover pack: what each file does, where the configurable rules live, how to update them safely, how the document-generation logic works, how the tool is deployed, and what tests should be run after a change.

This does not remove the need for human review. The builder still validates that the documentation reflects the real application. But it dramatically lowers the cost of producing documentation that is usually skipped because the project is moving too fast. For internal tools, this is not administrative polish; it is part of the value case. A tool that only its builder understands recreates the key-person risk it was meant to eliminate.

EXHIBIT 17 **Claude-assisted technical handover pack**

DOCUMENT	PURPOSE	WHAT CLAUDE CAN DRAFT
Architecture note	Explain how the app is structured	File map, main modules, data flow, dependency list
Configuration guide	Let non-developers update changing rules	JSON schema, editable fields, examples, validation rules
Maintenance manual	Support future fixes and ownership transfer	Common change procedures, test checklist, known limitations
Deployment guide	Make the tool redeployable without the original builder	Hosting steps, file locations, rollback procedure

```
"Here is the final codebase of my client-side tool. Create a technical handover document for a future maintainer. Include: file-by-file architecture, data schema, configurable fields, deployment steps, common update procedures, tests to run after a change, known limitations, and a short non-technical summary for the business owner."
```

PART VI

VI
Deployment & Operations

A tool nobody can reach creates no value. This part covers the full deployment spectrum and long-term operations.

6.1 — The deployment spectrum

EXHIBIT 18 **Deployment options — from free to enterprise**

OPTION	COST	SETUP	BEST FOR
GitHub Pages	Free	10 min	Static client-side tools
Netlify / Vercel	Free tier	15 min	Static + serverless functions, CI/CD
Cloudflare Pages	Free tier	15 min	Global CDN, Workers for edge logic
AWS S3 + CloudFront	~€1–5/mo	1–2h	Enterprise-grade, access control
Internal / SharePoint	Existing	Variable	Regulated environments
Docker + VPS	€5–20/mo	2–4h	Backend-dependent tools, databases

6.2 — GitHub Pages: the default

1. Create a free repository on `github.com`. Set it to public.
2. Upload the tool's files. `index.html` plus data files.
3. Enable Pages under Settings → Pages.
4. Select the main branch and root folder.
5. Live at `username.github.io/repo-name/`.

6.3 — When you need more

- **Serverless functions** (Netlify Functions, Cloudflare Workers, AWS Lambda) for light backend logic.

- **Backend-as-a-Service** (Supabase, Firebase) for auth and database without server code.
- **Full deployment** (Docker on VPS, Railway, Render) when criticality justifies overhead.

6.4 — Operations maturity ladder

EXHIBIT 19 **Operations maturity ladder**

LEVEL	STAGE	WHAT IT ADDS
0	Live	Deployed and reachable; data externalized; basic validation
1	Maintainable	Versioned data, structure validation, documented update path
2	Resilient	Internalized assets, graceful degradation, offline capability
3	Integrated	Backend for multi-user data, system links, sign-on
4	Governed	Monitoring, access control, audit trail, formal ownership

PART VII

VII
Measuring Realized Value

A projected return is a promise; a realized return is a result.

7.1 — The benefits gap

Two failure modes: **adoption shortfall** (the tool works but part of the team doesn't use it) and **assumption drift** (volumes or savings differ from projections). Both are invisible unless you measure.

7.2 — Leading and lagging indicators

EXHIBIT 20 **Benefits-realization indicators**

TYPE	METRIC	TELLS YOU
Leading	% of jobs processed through the tool	Adoption, the biggest driver of realized value
Leading	Active users / total eligible	Reach across the team
Leading	Rejected-application rate	Whether quality is improving
Lagging	Average preparation time per job	The core time saving, measured
Lagging	Cumulative hours saved → € realized	Actual value vs. business case
Lagging	Onboarding time for new operator	Whether knowledge-encoding benefit landed

7.3 — Closing the loop

Each proven success makes the next business case easier. Measurement is how a one-off win becomes a track record.

PART VIII

VIII
Scaling to a Portfolio

One tool is a project. A repeatable ability to find friction and remove it is a capability, and capabilities compound.

8.1 — From one tool to a capability

The first tool's most valuable output is the proof that the method works here. The marginal cost of each subsequent tool falls.

8.2 — Portfolio sequencing

Wave 1 (prove): high-value, certain-to-succeed tools. **Wave 2** (build): reuse patterns, lower marginal cost. **Wave 3** (extend): harder problems credibility now permits.

8.3 — Governance guardrails

EXHIBIT 24 **Governance guardrails for AI-built tools**

DOMAIN	GUARDRAIL
Data & privacy	Classify what data each tool touches; keep personal data client-side or properly governed.
Human oversight	AI-generated code is reviewed by a human before production.
Ownership	Every tool has a named owner and maintainer.
Consistency	Shared patterns for data separation, deployment, documentation.
Transparency	Disclose where AI was used; keep the human accountable.

PART IX

IX
Ideation & Discovery with AI

Most of this report treats AI as a builder. But AI is equally powerful as a finder, a tool for discovering what to build, not just for building it.

9.1 — The discovery bottleneck

The hardest part of the method is not Part IV (building). It is Part I (assessing) and Part II (targeting). AI can accelerate every step, not by replacing judgment but by multiplying the speed at which hypotheses are generated, tested, and refined.

9.2 — Six AI-assisted discovery techniques

1. Process decomposition

```
"Here is how a permit application works in our organization: [describe].
Decompose this into a swimlane diagram. For each step, estimate whether it is
value-adding, necessary, or waste."
```

2. Analogical benchmarking

Ask Claude to identify organizations that have solved a similar problem. The output is a brainstorming accelerant, not definitive research.

3. Opportunity brainstorming

Share the friction register; ask Claude to rank opportunities by return, feasibility, and strategic value. Then challenge the ranking.

4. Sensitivity stress-testing

```
"Given 88 minutes saved per task, 534 tasks/year, €77/hour loaded cost: build
me a sensitivity table varying minutes saved (60–140) and annual volume (400–
700)."
```

5. Stakeholder simulation

```
"You are the CFO. I'm presenting this business case. What are your three
toughest questions?"
```

6. Data-schema ideation

Describe the domain; ask Claude to propose a data schema. Iterate: what fields are missing? What changes across the 24 administrations? What is stable?

9.3 — The meta-principle

AI replaces the blank page, not the human's judgment. The most valuable use in discovery is to generate **first drafts, structured questions, and hypotheses** that the human then validates. The human who arrives with a pre-structured process map, even a wrong one, learns ten times faster than the one who arrives with a blank notebook.

PART X

X
Security Considerations

A tool that saves €597K but leaks sensitive data or opens an attack surface has not created value. Security is a design input that belongs in Part III's checklist from the start.

10.1 — Threat model

AI-built tools face two categories of risk: risks common to all internal tools (access control, data leakage, injection attacks), and risks specific to AI involvement (code quality assumptions, supply-chain blind spots, over-trust in generated output).

10.2 — Client-side security

- **No secrets in client-side code.** API keys must never appear in served JavaScript.
- **Input sanitization.** Any user input rendered as HTML must be sanitized against XSS.
- **Content Security Policy (CSP).** Set a strict CSP to prevent unauthorized script execution.
- **Local storage sensitivity.** Do not store PII in localStorage without encryption.

10.3 — AI-specific security risks

Code quality assumptions

AI-generated code is plausible, not proven. Run dependency audits (`npm audit`, `pip audit`) and review security-critical paths line by line.

Supply-chain awareness

Before adding any dependency Claude suggests: verify it exists, check its last update, review maintenance status, and check for known CVEs.

Prompt injection

If the tool passes user input to an AI model, the input must be treated as untrusted. Never concatenate raw user input into prompts; use structured input formats.

10.4 — Data protection

- **Data locality.** Client-side execution can reduce third-party data transfer risk significantly. However, if personal data is handled, the organization should still document the data flow and validate the setup with its DPO or legal team.
- **Third-party services.** Document which data leaves the browser and to whom.
- **Export and retention.** Inform users about documents containing personal data.

10.5 — Security checklist

- ☐ No API keys, credentials, or secrets in client-side code
- ☐ User inputs sanitized before rendering or document injection
- ☐ Content Security Policy set and tested
- ☐ Dependencies audited for known vulnerabilities
- ☐ Libraries verified as current, maintained, and legitimate
- ☐ Data flows documented: what leaves the browser, to where
- ☐ LocalStorage used only for non-sensitive data
- ☐ AI-generated code reviewed on all critical paths
- ☐ HTTPS enforced for all hosted deployments
- ☐ If user input touches an AI model: prompt-injection safeguards in place

PART XI

XI
Principles & Playbook

The method, distilled. Twelve principles, reusable templates, a prompt library, and a glossary.

11.1 — Twelve principles

1. **Target expensive friction**: the biggest wins are boring, repetitive tasks.
2. **Measure before you build**: a quantified baseline is the only defence against solving the wrong problem.
3. **Price everything in money**: "saves time" is forgotten; "saves €597K" gets approved.
4. **Be conservative on purpose**: defending the lower bound makes the case credible.
5. **Human expertise is the moat**: AI writes code; knowing what to build is irreplaceably human.
6. **Separate data from logic**: the single decision that makes a tool maintainable by non-developers.
7. **Design for total cost of ownership**: the build is a day; operation is years.
8. **Co-construct for adoption**: tools fail on adoption, not technology. Build with, not for.
9. **Measure the realized return**: a projected benefit is a promise; instrument the tool to prove it.
10. **Turn the win into a capability**: the first tool's real output is proof the method works here.
11. **Understand your AI partner**: token economics, model strengths, and limits shape what you build.
12. **Secure by design**: a tool that leaks data has not created value.

11.2 — Build brief template

```
Context: I'm building [tool] for [user] who [situation/constraint]. Goal: [the one outcome that matters]. Constraints: [framework / format / dependencies / where it runs]. Scope now: [the single slice to build first]. Definition of done: [what "correct" looks like]. Build the slice above; we'll deepen it next. Ask if anything is ambiguous.
```

11.3 — Prompt library

PROMPT LIBRARY **Ready-to-use templates**

WHEN YOU WANT TO…	TRY A PROMPT SHAPED LIKE
Start a build	"I'm building X for Y who must Z. Constraints: … Build the first slice: …"
Choose architecture	"Give me 2–3 options for how to store this data, with trade-offs."
Add a feature	"Now add [feature]. Change only that; leave the rest working."
Fix a precise issue	"On a real case, [behaviour] was wrong; it should [expectation]. Fix just that."
Make maintainable	"Move all changing rules into a separate JSON file; show me the schema."
Harden it	"Add validation for [fields]; if [service] fails, let the user proceed manually."
Optimize tokens	"Show me only the changed function, not the entire file."
Discover friction	"Here is how process X works: [describe]. Decompose into swimlane."
Stress-test a case	"You are the CFO. What are your three toughest questions?"
Security review	"Review this code for: XSS, exposed secrets, unsafe dependencies."

11.4 — Glossary

TERM	DEFINITION
Loaded hourly rate	Total cost of an hour of labour: salary plus employer charges and overhead.
Cost of friction	The annual monetary cost of wasted time in a process.
Swimlane map	A process diagram with one lane per actor, exposing handoffs.
SIPOC	Suppliers–Inputs–Process–Outputs–Customers; a frame bounding a process.
Value bridge	A waterfall from gross benefit through costs to net value created.
Total cost of ownership	Build effort plus all running, update, and hand-over costs across a tool's life.

Client-side	Software running entirely in the user's browser, with no server.
Data–logic separation	Keeping changing data apart from code, so non-developers can maintain it.
Vertical slice	A thin, working end-to-end version, built before deepening any feature.
Champion	An early pilot user whose visible success persuades the majority to adopt.
Token	The unit of AI consumption; roughly 0.75 words in English.
Context window	The maximum combined input + output an AI model can process in one call.
Prompt injection	An attack where malicious input overrides a model's instructions.
CSP	Content Security Policy: browser-level restriction on executable scripts.
GitHub Pages	A free service hosting static websites from a code repository.

> *The barrier to creating six-figure value inside most organizations is no longer engineering capacity. It is the willingness to look closely at a tedious process, measure it honestly, direct an AI to remove the friction, bring the team along so the result endures, and prove the return so the next one is easier. That work is within reach of one capable person, and it compounds. This report is how.*